

Part 1: The Phantom Bug

- Write a program that tosses $50,000,000$ random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.
- **Deliverable:** Run it 5 times. Record the value of π .
- *The Lesson:* The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race).

```
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % java PiRace
Total Hits: 26461328
Estimated Pi: 2.11690624
Time: 994.596083 ms
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % java PiRace
Total Hits: 26628068
Estimated Pi: 2.13024544
Time: 930.747333 ms
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % java PiRace
Total Hits: 22808806
Estimated Pi: 1.82470448
Time: 1004.714125 ms
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % java PiRace
Total Hits: 26622074
Estimated Pi: 2.12976592
Time: 932.082792 ms
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % java PiRace
Total Hits: 22737252
Estimated Pi: 1.81898016
Time: 1009.20425 ms
(base) dimasaltynbek@MacBook-Pro-Dimas desktop %
```

Run	Total Hits	Estimated π	Time [ms]
1	26,461,328	2.1169	994.60
2	26,628,068	2.1302	930.75
3	22,808,806	1.8247	1004.71
4	26,622,074	2.1298	932.08
5	22,737,252	1.8190	1009.20

Part 2: The Synchronization Trap

- Fix the race condition by adding synchronized to the increment or using AtomicLong.incrementAndGet().
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

```
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % javac PiFixed.java
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % java PiFixed.java
=== Single-threaded baseline ===
Pi: 3.14137728
Time: 1605.171792 ms

=== synchronized version ===
Pi: 3.14154688
Time: 4666.781375 ms

=== AtomicLong version ===
Pi: 3.141556
Time: 1222.068916 ms
(base) dimasaltynbek@MacBook-Pro-Dimas desktop %
```

Version	π	Time [ms]	vs Single-thread
Single-threaded baseline	3.14138	1605.17	1.00x
synchronized	3.14155	4666.78	2.91x slower
AtomicLong	3.14156	1222.07	1.31x faster

Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for $T \in \{1, 2, 4, 8, 16, 32\}$ threads on their machine with 100,000,000 iterations.
- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (T/N T1)	Efficiency (TSpeedup)
1	3222	1.0x	100%
2	1763	1.83x	91.4%

4	869	3.71x	92.7%
8	601	5.36x	67.0%
16	700	4.60x	28.8%
32	604	5.33x	16.7%

```
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % javac PiReduction.java
(base) dimasaltynbek@MacBook-Pro-Dimas desktop % java PiReduction
Threads | Runtime(ms) | Hits | Pi
-----|-----|-----|-----
1 | 3222 | 78541860 | 3.141674 | Speedup: 1.00x | Efficiency: 100.0%
2 | 1763 | 78541752 | 3.141670 | Speedup: 1.83x | Efficiency: 91.4%
4 | 869 | 78539700 | 3.141588 | Speedup: 3.71x | Efficiency: 92.7%
8 | 601 | 78535366 | 3.141415 | Speedup: 5.36x | Efficiency: 67.0%
16 | 700 | 78535438 | 3.141418 | Speedup: 4.60x | Efficiency: 28.8%
32 | 604 | 78537851 | 3.141514 | Speedup: 5.33x | Efficiency: 16.7%
```

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

So my 8-core CPU (M1: 4 Performance + 4 Efficiency cores) already had all its physical cores busy at T=8, and that's actually where I got my best time (601ms). Going to T=16 doesn't give me 16 physical cores to run on — I still only have 8. So the OS just has to time-slice 16 threads across 8 cores instead, which means each core is now context-switching between 2 threads instead of running 1 thread flat out.

That's why T=16 was actually slower (700ms) than T=8, not faster. Context switching isn't free — every time the OS swaps which thread a core is running, it has to save/restore registers and thread state, and on top of that Java has to spin up 16 Random objects and coordinate 16 thread creations/joins instead of 8, which adds its own overhead. So instead of getting more real work done in parallel, I'm just paying more scheduling tax for the same 8 physical cores.

2. *Why was the synchronized version in Part 2 slower than running on one single core?*

Because synchronized forces every single thread to grab the same lock before it can do `totalHitsSync++`, and only one thread is allowed to hold that lock at a time. So even though I had 4 threads technically running "in parallel," they all had to line up and take turns doing the actual increment — the increment itself became fully serial again, exactly like running on one thread, except now I've also added a bunch of extra overhead on top: acquiring/releasing the lock 50,000,000 times total, plus threads potentially getting suspended and woken back up by the OS when they can't get the lock right away (that's the "cores spend 95% of their cycles stalling on memory bus locks" part from the assignment).

So it's not just "no speedup" — it's actively worse than 1 thread, because now I'm paying for thread creation/coordination AND lock contention overhead, but getting zero benefit from parallelism since the increments still happen one at a time anyway.

Upload your solutions before the class end-time to the Git repo that you created for benchmarks. Share repo access with @sufyanism (GitHub/GitLab).